

Paso 23C — una fila por party con Prior 9m y Recent 3m

Ahora sí podemos construir la tabla de ventanas. Primero agregamos `weighted_utilization` al party-month mediante join y después resumimos los 12 meses.

```
DROP TABLE IF EXISTS
```

```
bu_frd.model_party_behavior_windows_2022_2025;
```

```
CREATE TABLE
```

```
bu_frd.model_party_behavior_windows_2022_2025
```

```
USING PARQUET
```

```
AS
```

```
WITH monthly AS (
```

```
  SELECT
```

```
    b.*,
```

```
    u.weighted_utilization,
```

```
    /* Inputs capped ONLY for Z-score challenger */
```

```
    CASE
```

```
      WHEN b.avg_utilization IS NULL
```

```
        THEN NULL
```

```
      ELSE LEAST(b.avg_utilization, 1.821143)
```

```
    END AS avg_utilization_z_input,
```

```
    CASE
```

```
      WHEN u.weighted_utilization IS NULL
```

```
        THEN NULL
```

```
      ELSE LEAST(u.weighted_utilization, 1.75333)
```

```
    END AS weighted_utilization_z_input
```

```
FROM
```

```
  bu_frd.model_party_behavior_monthly_2022_2025 b
```

```
LEFT JOIN
```

```
  bu_frd.model_party_utilization_monthly_2022_2025 u
```

```
  ON b.party_id = u.party_id
```

```
  AND b.observation_date = u.observation_date
```

```
  AND b.feature_month = u.feature_month
```

```
),
```

agg AS (

SELECT

party_id,
observation_date,
observation_year,
model_split,
label_3m,
cohort_role,

/* =====
COUNTS: DETERIORATION
===== */

SUM(
CASE WHEN feature_window = 'RECENT_3M'
THEN deterioration_count ELSE 0 END
) AS deterioration_recent_3m,

SUM(
CASE WHEN feature_window = 'PRIOR_9M'
THEN deterioration_count ELSE 0 END
) AS deterioration_prior_9m,

/* Monthly baseline mean + SD for Z-score */

AVG(
CASE WHEN feature_window = 'PRIOR_9M'
THEN deterioration_count END
) AS deterioration_prior_monthly_avg,

STDDEV_SAMP(
CASE WHEN feature_window = 'PRIOR_9M'
THEN deterioration_count END
) AS deterioration_prior_monthly_sd,

/* =====
ACTIVE ADVERSE
===== */

SUM(
CASE WHEN feature_window = 'RECENT_3M'
THEN adverse_entry_count ELSE 0 END
) AS adverse_recent_3m,

```
SUM(  
    CASE WHEN feature_window = 'PRIOR_9M'  
        THEN adverse_entry_count ELSE 0 END  
) AS adverse_prior_9m,
```

```
AVG(  
    CASE WHEN feature_window = 'PRIOR_9M'  
        THEN adverse_entry_count END  
) AS adverse_prior_monthly_avg,
```

```
STDDEV_SAMP(  
    CASE WHEN feature_window = 'PRIOR_9M'  
        THEN adverse_entry_count END  
) AS adverse_prior_monthly_sd,
```

```
/* =====  
ACTIVE ACCOUNTS  
===== */
```

```
AVG(  
    CASE WHEN feature_window = 'RECENT_3M'  
        THEN active_accounts END  
) AS avg_active_accounts_recent_3m,
```

```
AVG(  
    CASE WHEN feature_window = 'PRIOR_9M'  
        THEN active_accounts END  
) AS avg_active_accounts_prior_9m,
```

```
STDDEV_SAMP(  
    CASE WHEN feature_window = 'PRIOR_9M'  
        THEN active_accounts END  
) AS active_accounts_prior_sd,
```

```
/* =====  
BALANCE  
===== */
```

```
AVG(  
    CASE WHEN feature_window = 'RECENT_3M'  
        THEN avg_balance_active END  
) AS avg_balance_recent_3m,
```

```
AVG(  
    CASE WHEN feature_window = 'PRIOR_9M'  
        THEN avg_balance_active END
```

```

) AS avg_balance_prior_9m,

STDDEV_SAMP(
  CASE WHEN feature_window = 'PRIOR_9M'
    THEN avg_balance_active END
) AS balance_prior_sd,

COUNT(
  CASE
    WHEN feature_window = 'RECENT_3M'
    AND avg_balance_active IS NOT NULL
    THEN 1
  END
) AS balance_recent_months_available,

COUNT(
  CASE
    WHEN feature_window = 'PRIOR_9M'
    AND avg_balance_active IS NOT NULL
    THEN 1
  END
) AS balance_prior_months_available,

/* =====
PAST DUE
===== */

AVG(
  CASE WHEN feature_window = 'RECENT_3M'
    THEN avg_past_due_active END
) AS avg_past_due_recent_3m,

AVG(
  CASE WHEN feature_window = 'PRIOR_9M'
    THEN avg_past_due_active END
) AS avg_past_due_prior_9m,

STDDEV_SAMP(
  CASE WHEN feature_window = 'PRIOR_9M'
    THEN avg_past_due_active END
) AS past_due_prior_sd,

COUNT(
  CASE
    WHEN feature_window = 'RECENT_3M'
    AND avg_past_due_active IS NOT NULL
    THEN 1

```

```

        END
    ) AS past_due_recent_months_available,

    COUNT(
        CASE
            WHEN feature_window = 'PRIOR_9M'
            AND avg_past_due_active IS NOT NULL
            THEN 1
        END
    ) AS past_due_prior_months_available,

    /* =====
    DELINQUENCY SHARE
    ===== */

    AVG(
        CASE WHEN feature_window = 'RECENT_3M'
            THEN delinquent_account_share END
    ) AS delinquency_share_recent_3m,

    AVG(
        CASE WHEN feature_window = 'PRIOR_9M'
            THEN delinquent_account_share END
    ) AS delinquency_share_prior_9m,

    STDDEV_SAMP(
        CASE WHEN feature_window = 'PRIOR_9M'
            THEN delinquent_account_share END
    ) AS delinquency_share_prior_sd,

    COUNT(
        CASE
            WHEN feature_window = 'RECENT_3M'
            AND delinquent_account_share IS NOT NULL
            THEN 1
        END
    ) AS delinquency_recent_months_available,

    COUNT(
        CASE
            WHEN feature_window = 'PRIOR_9M'
            AND delinquent_account_share IS NOT NULL
            THEN 1
        END
    ) AS delinquency_prior_months_available,

```

```
/* =====  
AVG ACCOUNT UTILIZATION - RAW  
===== */
```

```
AVG(  
  CASE WHEN feature_window = 'RECENT_3M'  
    THEN avg_utilization END  
) AS avg_utilization_recent_3m,
```

```
AVG(  
  CASE WHEN feature_window = 'PRIOR_9M'  
    THEN avg_utilization END  
) AS avg_utilization_prior_9m,
```

```
COUNT(  
  CASE  
    WHEN feature_window = 'RECENT_3M'  
      AND avg_utilization IS NOT NULL  
    THEN 1  
  END  
) AS utilization_recent_months_available,
```

```
COUNT(  
  CASE  
    WHEN feature_window = 'PRIOR_9M'  
      AND avg_utilization IS NOT NULL  
    THEN 1  
  END  
) AS utilization_prior_months_available,
```

```
/* Capped input only for Z */
```

```
AVG(  
  CASE WHEN feature_window = 'PRIOR_9M'  
    THEN avg_utilization_z_input END  
) AS utilization_z_prior_avg,
```

```
AVG(  
  CASE WHEN feature_window = 'RECENT_3M'  
    THEN avg_utilization_z_input END  
) AS utilization_z_recent_avg,
```

```
STDDEV_SAMP(  
  CASE WHEN feature_window = 'PRIOR_9M'  
    THEN avg_utilization_z_input END  
) AS utilization_prior_sd,
```

```

/* =====
WEIGHTED UTILIZATION
===== */

AVG(
  CASE WHEN feature_window = 'RECENT_3M'
        THEN weighted_utilization END
) AS weighted_utilization_recent_3m,

AVG(
  CASE WHEN feature_window = 'PRIOR_9M'
        THEN weighted_utilization END
) AS weighted_utilization_prior_9m,

COUNT(
  CASE
    WHEN feature_window = 'RECENT_3M'
      AND weighted_utilization IS NOT NULL
    THEN 1
  END
) AS weighted_util_recent_months_available,

COUNT(
  CASE
    WHEN feature_window = 'PRIOR_9M'
      AND weighted_utilization IS NOT NULL
    THEN 1
  END
) AS weighted_util_prior_months_available,

AVG(
  CASE WHEN feature_window = 'PRIOR_9M'
        THEN weighted_utilization_z_input END
) AS weighted_util_z_prior_avg,

AVG(
  CASE WHEN feature_window = 'RECENT_3M'
        THEN weighted_utilization_z_input END
) AS weighted_util_z_recent_avg,

STDDEV_SAMP(
  CASE WHEN feature_window = 'PRIOR_9M'
        THEN weighted_utilization_z_input END
) AS weighted_util_prior_sd,

/* =====

```

```

MAX RECENT DPD JUMP
===== */

MAX(
  CASE WHEN feature_window = 'RECENT_3M'
    THEN max_dpd_jump END
) AS max_dpd_jump_recent_3m

FROM monthly

GROUP BY
  party_id,
  observation_date,
  observation_year,
  model_split,
  label_3m,
  cohort_role
)

SELECT *
FROM agg;

```

QA del 23C

Primero debe conservar exactamente una fila por persona:

```

SELECT
  COUNT(*) AS rows,
  COUNT(DISTINCT party_id) AS parties

FROM bu_frd.model_party_behavior_windows_2022_2025;

```

Esperamos:

688,589 / 688,589

Luego revisemos disponibilidad de las ventanas:

```

SELECT
  model_split,
  label_3m,

  COUNT(*) AS parties,

  ROUND(
    AVG(utilization_recent_months_available),
    2
  ) AS avg_util_recent_months,

```

```

ROUND(
  AVG(utilization_prior_months_available),
  2
) AS avg_util_prior_months,

ROUND(
  AVG(balance_recent_months_available),
  2
) AS avg_balance_recent_months,

ROUND(
  AVG(balance_prior_months_available),
  2
) AS avg_balance_prior_months

FROM bu_frd.model_party_behavior_windows_2022_2025

GROUP BY
  model_split,
  label_3m

ORDER BY
  model_split,
  label_3m DESC;

```

Paso 23D — fórmula actual

Después construiremos las features V1.

```

DROP TABLE IF EXISTS
bu_frd.model_behavior_features_current_2022_2025;

CREATE TABLE
bu_frd.model_behavior_features_current_2022_2025
USING PARQUET
AS

SELECT
  *,

  /* Event-rate acceleration */

  deterioration_recent_3m / 3.0
  - deterioration_prior_9m / 9.0
  AS deterioration_acceleration,

```

```

adverse_recent_3m / 3.0
- adverse_prior_9m / 9.0
  AS adverse_acceleration,

/* Continuous Recent - Baseline */

avg_active_accounts_recent_3m
- avg_active_accounts_prior_9m
  AS active_accounts_change_vs_baseline,

avg_balance_recent_3m
- avg_balance_prior_9m
  AS balance_change_vs_baseline,

avg_past_due_recent_3m
- avg_past_due_prior_9m
  AS past_due_change_vs_baseline,

delinquency_share_recent_3m
- delinquency_share_prior_9m
  AS delinquency_share_change_vs_baseline,

avg_utilization_recent_3m
- avg_utilization_prior_9m
  AS utilization_change_vs_baseline,

weighted_utilization_recent_3m
- weighted_utilization_prior_9m
  AS weighted_utilization_change_vs_baseline

FROM
  bu_frd.model_party_behavior_windows_2022_2025;

```

Esas son nuestras **features V1 oficiales**.

Paso 23E — Z-score challenger

Aquí mantendría una regla conservadora:

Para Z-score continuo necesitamos:

prior SD > 0

y además al menos:

2 meses históricos disponibles

para que la desviación estándar tenga significado.

Para utilization además exigiría inicialmente:

recent availability = 3

porque no quiero comparar un promedio de un solo mes reciente contra un baseline histórico y llamarlo “3-month Z-score”.

La fórmula será:

$$Z = \frac{\text{RecentMean} - \text{PriorMean}}{\text{PriorSD} / \sqrt{3}}$$

Para deterioration y adverse, como tienen los 9+3 meses completos, podemos usar directamente:

$$Z = \frac{\text{RecentCount} - 3 \times \text{PriorMonthlyMean}}{\text{PriorSD} \sqrt{3}}$$

El próximo SQL será aproximadamente:

deterioration_z

adverse_z

active_accounts_z

balance_z

past_due_z

delinquency_share_z

utilization_z

weighted_utilization_z

+ zero_variance flags

Pero antes de escribirlo quiero obtener **un último diagnóstico de los SD.**

Ejecuta:

```
SELECT
  model_split,
  label_3m,

  COUNT(*) AS parties,

  /* Deterioration */
```

```

SUM(
    CASE WHEN deterioration_prior_monthly_sd = 0
        THEN 1 ELSE 0 END
) AS deterioration_sd_zero,

ROUND(
    100.0 *
    SUM(
        CASE WHEN deterioration_prior_monthly_sd = 0
            THEN 1 ELSE 0 END
    ) / COUNT(*),
    2
) AS deterioration_sd_zero_pct,

```

/* Adverse */

```

SUM(
    CASE WHEN adverse_prior_monthly_sd = 0
        THEN 1 ELSE 0 END
) AS adverse_sd_zero,

ROUND(
    100.0 *
    SUM(
        CASE WHEN adverse_prior_monthly_sd = 0
            THEN 1 ELSE 0 END
    ) / COUNT(*),
    2
) AS adverse_sd_zero_pct,

```

/* Utilization */

```

SUM(
    CASE WHEN utilization_prior_sd > 0
        THEN 1 ELSE 0 END
) AS utilization_valid_sd,

ROUND(
    100.0 *
    SUM(
        CASE WHEN utilization_prior_sd > 0
            THEN 1 ELSE 0 END
    ) / COUNT(*),
    2
) AS utilization_valid_sd_pct,

```

```

/* Weighted utilization */

SUM(
  CASE WHEN weighted_util_prior_sd > 0
    THEN 1 ELSE 0 END
) AS weighted_util_valid_sd,

ROUND(
  100.0 *
  SUM(
    CASE WHEN weighted_util_prior_sd > 0
      THEN 1 ELSE 0 END
    ) / COUNT(*),
  2
) AS weighted_util_valid_sd_pct

FROM
  bu_frd.model_party_behavior_windows_2022_2025

GROUP BY
  model_split,
  label_3m

ORDER BY
  model_split,
  label_3m DESC;

```

Ese resultado nos va a responder directamente una de las preguntas que teníamos desde el principio:

¿En qué variables el Z-score es realmente viable con nuestros datos?

Si deterioration tiene, por ejemplo, 90%+ SD=0, no tiene sentido forzarlo con un floor artificial. En cambio, si utilization tiene buena cobertura de SD > 0, entonces ahí el challenger Z-score sí será estadísticamente mucho más interesante.

Con esos resultados cerramos 23C/23D y construimos **23E: Z-score challenger**.